

13주차

웹과 DB 연동 2 (CRUD)

폼에서 INSERT까지, 트랜잭션 주문 처리

한국기술교육대학교 경영학부

경영데이터베이스및실습 (CCT770) · 2026학년도 2학기

이번 주를 마치면

목표 ①	CRUD 네 기능과 SQL·HTTP 요청의 대응 관계 설명
목표 ②	등록 폼에서 받은 값을 INSERT로 저장하는 라우트 만들기
목표 ③	UPDATE·DELETE 라우트를 작성하고 affectedRows로 결과 확인
목표 ④	주문 등록처럼 여러 SQL이 한 묶음인 작업에 트랜잭션 적용
목표 ⑤	WHERE 없는 UPDATE/DELETE 금지 등 웹-DB 안전 수칙 실천

12주차 app.js에 라우트를 이어 붙여 CRUD를 완성합니다

오늘 다루는 여섯 가지

1 CRUD — 업무 화면의 네 가지 기본 동작

2 등록 — 폼에서 INSERT까지

3 수정과 삭제

4 CRUD 전체 흐름 정리

5 트랜잭션이 필요한 순간 — 주문 등록

6 웹-DB 안전 수칙

1. CRUD

업무 화면의 네 가지 기본 동작

기능	SQL	HTTP 요청	다운유통 예
Create (등록)	INSERT	POST	신규 고객 등록
Read (조회)	SELECT	GET	상품 목록·검색 (12주차)
Update (수정)	UPDATE	POST (실무 API는 PUT)	상품 재고 수정, 주문 취소
Delete (삭제)	DELETE	POST (실무 API는 DELETE)	주문 상세에서 상품 한 줄 빼기

회원 관리·재고 관리·주문 관리 — 어떤 관리자 화면도 결국 이 네 가지 동작의 조합

1. CRUD

이번 실습의 두 가지 약속

수정·삭제도 POST로

01

브라우저 <form>은 GET과 POST만 보낼 수 있다
실무 REST API는 PUT·DELETE를 구분해 쓴다

12주차 app.js에 이어 붙이기

02

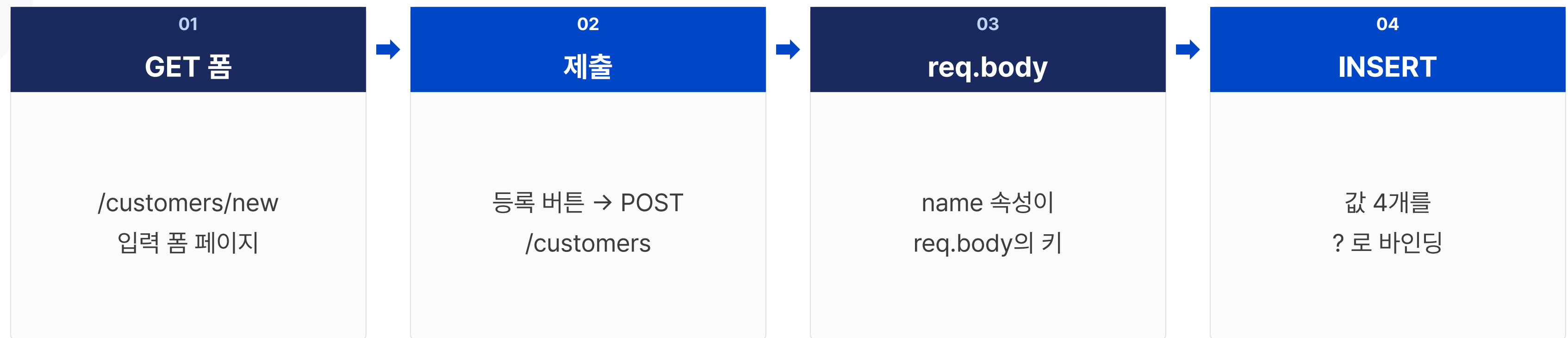
pool(커넥션 풀)과 서버 실행 코드는 그대로
새 라우트는 app.listen(...) 윗줄에 추가

PUT·DELETE 메서드는 개념만 알아 두면 된다

2. 등록

등록은 두 단계 — 폼 보여 주기와 저장하기

폼의 각 `<input name="...">` 값은 서버에서 `req.body.이름` 으로 꺼낸다



같은 주소 `/customers`라도 `app.get`이 아니라 `app.post`이므로 다른 라우트다

2. 등록

준비 한 결과 ① 고객 등록 폼 — GET /customers/new

```
1 // 폼이 POST로 보낸 값(요청 본문)을 해석해 req.body 에 담아 주는 설정
2 // 이 줄이 없으면 req.body 가 undefined 라서 req.body.name 에서 에러가 난다
3 // app.use(...) 는 "모든 요청에 공통으로 적용" - 라우트들보다 위에 써야 한다
4 app.use(express.urlencoded({ extended: true }));
5
6 app.get('/customers/new', (req, res) => {
7   res.send(
8     '<meta charset="utf-8"><h1>신규 고객 등록</h1>' +
9     '<form method="POST" action="/customers">' +
10    '<고객번호: <input name="cust_id" placeholder="C011"><br>' +
11    '<이름: <input name="name"><br>' +
12    '<도시: <input name="city"><br>' +
13    '<이메일: <input name="email"><br>' +
14    '<button type="submit">등록</button>' +
15    '</form>'
16  );
17 });
```

1

1-4행

1 POST 본문 읽기 설정

없으면 req.body가 undefined — 라우트보다 위에

2

9행

2 method·action

입력값을 본문에 담아 /customers로

3

10-13행

3 input의 name

서버에서 req.body.이름의 '이름'

4

14행

4 submit 버튼

누르면 폼 내용을 POST 전송

1~4행은 app.js 위쪽에 · 폼 라우트는 DB를 쓰지 않고 HTML만 돌려주므로 async가 없다

2. 등록

② 폼이 보낸 값을 받아 INSERT — POST /customers

```
1 app.post('/customers', async (req, res) => {
2   try {
3     await pool.query(
4       'INSERT INTO customers (cust_id, name, city, email, joined_date) VALUES (?, ?, ?, ?, CURDATE())',
5       [req.body.cust_id, req.body.name, req.body.city, req.body.email] // ? 순서 = 컬럼 순서
6     );
7     res.send('<meta charset="utf-8">등록 완료! <a href="/customers/new">계속 등록</a>');
8   } catch (err) {
9     res.status(500).send('등록 실패: ' + err.message);
10  }
11 });
```

INSERT는 결과 행이 없으므로 [rows]로 받지 않고 완료만 기다린다(await)

2. 등록

등록 라우트에서 짚을 네 가지

01

? 바인딩 필수

폼 입력은 사용자가
마음대로 쓰는 값

02

가입일

사용자에게 받지 않고
CURDATE()로 채운다

03

grade·points

INSERT에서 뺐다
DEFAULT SILVER·0

04

? 순서

배열 순서 =
컬럼 순서

grade·points의 DEFAULT는 3주차 DDL에서 정한 값이 자동으로 채워 준다

2. 등록

등록할 때 만나는 에러

■ 증상

! req.body.name에서 에러

! ER_DUP_ENTRY 에러

! 등록 실패: ...

■ 원인

req.body가 undefined

email의 UNIQUE 제약 위반 (예: garam@example.com)

중복 이메일.길이 초과 등 DB가 거부

■ 해결

✓ app.use(express.urlencoded(...))
를 라우트보다 위에 둔다

✓ 에러 메시지를 생성형 AI에 붙여 '무슨
뜻이야?'라고 물어본다

✓ catch에서 500으로 응답 — 메시지로
원인을 확인한다

일부러 한 번 중복 이메일로 등록해 보기 — 제약조건이 잘못된 데이터를 막아 주는 순간을 직접 본다

3. 수정과 삭제

3.1 상품 재고 수정 ① 폼 — GET /stock

```
1 // ① 수정 폼 - http://localhost:3000/stock
2 app.get('/stock', (req, res) => {
3   res.send(
4     '<meta charset="utf-8"><h1>상품 재고 수정</h1>' +
5     '<form method="POST" action="/stock">' +           // 제출하면 아래 ② 로 POST 전송
6     '상품번호: <input name="prod_id" placeholder="P004"><br>' + // 어느 상품을 고칠지
7     '새 재고 수량: <input name="stock" type="number"><br>' +    // 새 값. type=number 는 숫자 키패드 안내용
8     '<button type="submit">수정</button>' +
9     '</form>'
10  );
11 });
```

입고·실사 후 재고 수량을 바로잡는 화면 — 대상(prod_id)과 새 값(stock)을 받는다

3. 수정과 삭제

3.1 상품 재고 수정 ② UPDATE — POST /stock

```
1 // ② UPDATE 실행
2 app.post('/stock', async (req, res) => {
3   try {
4     const [result] = await pool.query(
5       'UPDATE products SET stock = ? WHERE prod_id = ?',
6       [req.body.stock, req.body.prod_id]
7     );
8     res.send('<meta charset="utf-8">수정된 행 수: ' + result.affectedRows);
9   } catch (err) {
10    res.status(500).send('수정 실패: ' + err.message);
11  }
12 });
```

4행

1 **result로 받는다**

행 배열이 아니라 처리 결과 객체

5-6행

2 **? 순서**

SET의 ?가 첫째, WHERE의 ?가 둘째

8행

3 **affectedRows**

실제로 바뀐 행 수 — 0이면 없는 상품번호

WHERE를 빠뜨리면 전 상품 재고가 한 값으로 바뀐다 — 절대 금지

affectedRows — 결과를 판단하는 근거

처리 결과 객체

01

INSERT·UPDATE·DELETE의 결과는
행 배열이 아니다

0이면

02

WHERE에 걸린 행이 없었다
→ 「해당 상품이 없습니다」

직접 해 보기

03

품절이던 P004(제주 감귤)에
재고를 입고 처리

SELECT의 결과는 rows(행 배열), 변경 SQL의 결과는 result.affectedRows(바뀐 행 수)

3.2 주문 취소 — DELETE가 아니라 UPDATE

기록성 데이터

01

주문은 매출·반품·고객 응대의
근거 — 지우면 안 된다

상태만 바꾼다

02

시드 데이터의 주문 1005·1013이
status = '취소'로 남아 있다

경영 질문에 답한다

03

기록이 남아야 「이번 달 취소율이
왜 올랐지?」에 답할 수 있다

행을 지우지 않고(DELETE 금지) status 컬럼만 바꿔 기록을 남긴다

3. 수정과 삭제

주문 취소 처리 — POST /orders/cancel

```
1 app.post('/orders/cancel', async (req, res) => {
2   try {
3     const [result] = await pool.query(
4       "UPDATE orders SET status = '취소' WHERE order_id = ?",
5       [req.body.order_id]
6     );
7     if (result.affectedRows === 0) {
8       res.send('해당 주문이 없습니다. ');
9     } else {
10      res.send('<meta charset="utf-8">주문이 취소 처리되었습니다. ');
11    }
12  } catch (err) {
13    res.status(500).send('취소 실패: ' + err.message);
14  }
15 });
```

4행

1 큰따옴표로 감싼 SQL

안에 '취소'라는 작은따옴표가 있어서

1

2

5행

2 입력값만 ?

'취소'는 고정 값, order_id만 바인딩

3

7-11행

3 affectedRows 분기

0이면 그 주문번호의 행이 없었다

3. 수정과 삭제

3.3 진짜 삭제가 필요할 때 — DELETE

```
1 app.post('/order-items/delete', async (req, res) => {
2   try {
3     const [result] = await pool.query(
4       'DELETE FROM order_items WHERE order_id = ? AND prod_id = ?',
5       [req.body.order_id, req.body.prod_id]
6     );
7     res.send('삭제된 행 수: ' + result.affectedRows);
8   } catch (err) {
9     res.status(500).send('삭제 실패: ' + err.message);
10  }
11 });
```

4행

- 1 복합 키 두 조건
(order_id, prod_id)를 AND로 함께

5행

- 2 ? 두 개 ↔ 두 칸
하나만 걸면 그 주문의 상품이 전부 지워진다

7행

- 3 결과 확인
정상이면 1, 해당 줄이 없으면 0

customers의 고객 행은 주문이 남아 있는 한 외래키(FK) 제약 에러로 DELETE가 거부된다

이번 주 실습 라우트 한눈에

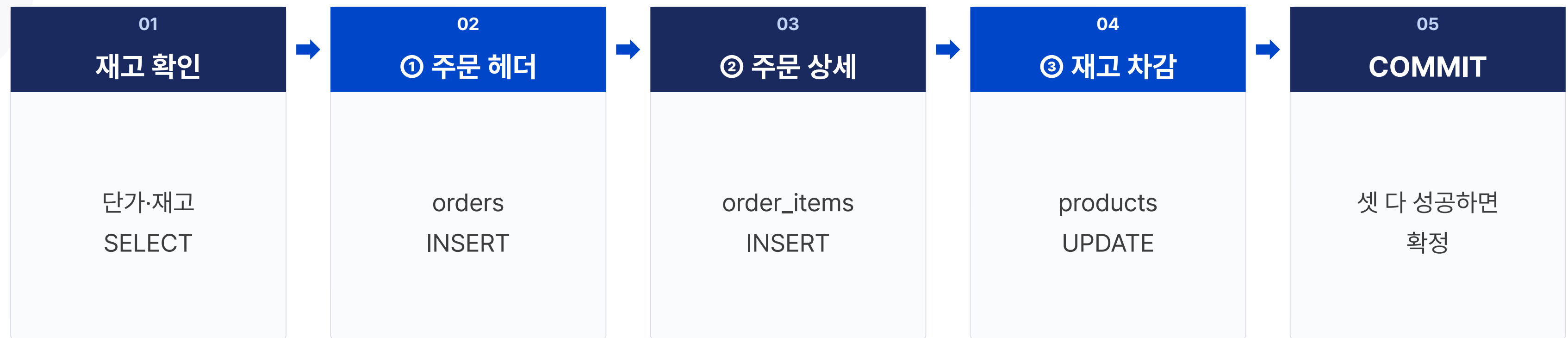
기능	라우트	핵심 SQL	성공 확인
등록	POST /customers	INSERT INTO customers ... VALUES (?, ...)	에러 없이 완료 (실패 시 catch)
조회	GET /products · /search	SELECT ... WHERE ... (12주차)	rows 배열의 내용
수정	POST /stock · /orders/cancel	UPDATE ... SET ... WHERE ?	affectedRows ≥ 1
삭제	POST /order-items/delete	DELETE FROM ... WHERE ?	affectedRows ≥ 1

orders(AUTO_INCREMENT)에 INSERT하면 방금 발급된 주문번호가 result.insertId로 돌아온다

5. 트랜잭션이 필요한 순간

주문 하나 = SQL 세 개가 한 묶음

②까지 성공하고 ③에서 서버가 멈추면? 주문은 있는데 재고는 그대로 — 장부가 안 맞는 상태



하나라도 실패하면 ROLLBACK으로 전부 되돌린다 — 4주차에 배운 트랜잭션

5. 트랜잭션이 필요한 순간

주문 등록 ① 주문 폼 — GET /order

```
1 // ① 주문 폼 - http://localhost:3000/order
2 app.get('/order', (req, res) => {
3   res.send(
4     '<meta charset="utf-8"><h1>주문 등록</h1>' +
5     '<form method="POST" action="/order">' +
6     '고객번호: <input name="cust_id" placeholder="C003"><br>' +
7     '상품번호: <input name="prod_id" placeholder="P001"><br>' +
8     '수량: <input name="qty" type="number" value="1"><br>' + // value="1" 은 기본값
9     '<button type="submit">주문</button>' +
10    '</form>'
11  );
12 });
```

고객번호·상품번호·수량을 받아 POST /order로 보낸다

5. 트랜잭션이 필요한 순간

주문 등록 ② 연결 꺼내기와 재고 확인

```
14 app.post('/order', async (req, res) => {
15   const conn = await pool.getConnection(); // 트랜잭션은 같은 연결에서 처리
16   try {
17     await conn.beginTransaction();
18
19     // 1) 상품의 현재 단가·재고 확인
20     const [prods] = await conn.query(
21       'SELECT price, stock FROM products WHERE prod_id = ?',
22       [req.body.prod_id]
23     );
24     if (prods.length === 0) throw new Error('없는 상품번호입니다. ');
25     if (prods[0].stock < req.body.qty) throw new Error('재고가 부족합니다. ');
```

1

2

3

4

15행

1 getConnection

풀에서 연결 하나를 꺼내 끝까지 그것만

17행

2 트랜잭션 시작

commit 전까지 변경은 임시 상태

20-23행

3 conn.query

꺼내 둔 그 연결로 실행해야 한 묶음

24-25행

4 throw

즉시 catch로 — 아래 INSERT 실행 안 됨

pool.query는 매번 다른 연결을 쓸 수 있어 트랜잭션에 부적합하다

5. 트랜잭션이 필요한 순간

주문 등록 ③ 헤더 · 상세 · 재고 차감

```
27  const [result] = await conn.query(  
28    'INSERT INTO orders (cust_id, order_date) VALUES (?, CURDATE())',  
29    [req.body.cust_id]  
30  );  
31  const orderId = result.insertId;  // 방금 발급된 주문번호  
32  
33  // 3) 주문 상세 - 주문 시점 단가를 함께 저장  
34  await conn.query(  
35    'INSERT INTO order_items (order_id, prod_id, qty, unit_price) VALUES (?, ?, ?, ?)',  
36    [orderId, req.body.prod_id, req.body.qty, prods[0].price]  
37  );  
38  
39  // 4) 재고 차감  
40  await conn.query(  
41    'UPDATE products SET stock = stock - ? WHERE prod_id = ?',  
42    [req.body.qty, req.body.prod_id]  
43  );
```

27-30행

- 1 2) 주문 헤더
order_id는 AUTO_INCREMENT — DB가 발급

31행

- 2 insertId
방금 발급된 주문번호로 상세와 연결

34-37행

- 3 주문 상세
그때 판 가격을 unit_price로 남긴다

40-43행

- 4 stock - ?
현재 값에서 주문 수량만큼 뺀다

웹 주문이므로 emp_id 없이(온라인 주문), status는 DEFAULT '주문접수'

5. 트랜잭션이 필요한 순간

주문 등록 ④ 확정 · 되돌리기 · 반납

```
45     await conn.commit(); // 여기까지 왔으면 전부 확정
46     res.send('<meta charset="utf-8">주문 완료! 주문번호: ' + orderId);
47   } catch (err) {
48     await conn.rollback(); // 하나라도 실패하면 전부 되돌린다
49     res.status(500).send('주문 실패: ' + err.message);
50   } finally {
51     conn.release(); // 연결을 풀에 반납 (성공/실패와 무관하게 필수)
52   }
53 });
```

1

45-46행

1 commit

여기까지 왔으면 전부 확정

2

3

47-49행

2 rollback

재고 부족 throw·DB 오류 → 전부 되돌림

50-51행

3 finally · release

성공하든 실패하든 연결을 풀에 반납

반납을 빠뜨리면 연결이 하나씩 새서 connectionLimit(10)에 닿는 순간 서버가 멈춘다

주문 등록 코드에서 꼭 남길 습관

재고 부족이면 throw

01

주문 자체가 없던 일이 된다
확인하고 확정 사이 빈틈을 메운다

웹 주문은 emp_id 없이

02

시드 데이터의 온라인 주문
(emp_id NULL)이 이렇게 생겼다

돈·재고 = 트랜잭션

03

주문·결제·이체처럼 돈과 재고가
움직이는 처리는 반드시 묶는다

재고만 줄고 주문은 없는 「유령 출고」가 쌓이면 월말 재고 실사가 장부와 어긋난다

6. 웹-DB 안전 수칙

코드를 실행하기 전 다섯 가지 확인



WHERE 없는 UPDATE/DELETE 금지

UPDATE products SET stock = 0 → 전 상품 품절 · 같은 WHERE로 먼저 SELECT



사용자 입력은 예외 없이 ? 바인딩

등록·수정·삭제는 입력값이 SQL에 직접 닿는 통로 (12주차)



기록성 데이터는 DELETE 대신 상태 변경

주문 취소는 status UPDATE로 이력을 남긴다



웹 서버용 DB 계정은 최소 권한

SELECT·INSERT·UPDATE 정도만 GRANT, DROP·ALTER는 주지 않는다



여러 SQL이 한 묶음이면 트랜잭션

하나라도 실패하면 전부 되돌아가야 장부가 맞는다

AI가 준 코드도 그대로 믿지 말고 직접 실행해 `affectedRows`와 화면으로 확인하는 것까지가 실습

실습 총괄보고서 안내 (20점)

일정

01

13~14주차에 작성
15주차에 제출

이번 주에 할 일

02

12~13주차 조회·CRUD 페이지의
코드와 실행 화면을 캡처해 정리

생성형 AI 활용

03

활용해도 좋다 — 단, 직접 실행해
확인한 코드만 심는다

14주차 경영 데이터 종합 실습 결과와 합쳐 보고서를 완성한다

확인 퀴즈

주문 등록 시 orders에 INSERT하고 이어서 order_items에 INSERT하는 두 작업을 하나의 트랜잭션으로 묶는 이유는?

① 실행 속도가 빨라지기 때문

② 한쪽만 성공하고 다른 쪽이 실패하는 불완전한 주문을 막기 위해

③ SQL 인젝션을 막기 위해

④ 커넥션 풀 사용량을 줄이기 위해

정답 및 해설

정답

② 한쪽만 성공하고 다른 쪽이 실패하는 불완전한 주문을 막기 위해

해설

주문 헤더만 있고 상세가 없는(또는 그 반대) 데이터는 매출 집계를 왜곡합니다. 트랜잭션의 원자성으로 전부 반영되거나 전부 취소되게 만듭니다.

13주차 핵심 정리

핵심 ①	업무 화면은 CRUD의 조합 — INSERT·SELECT·UPDATE·DELETE
핵심 ②	폼 값은 req.body로, app.use(express.urlencoded(...))가 필수
핵심 ③	변경 결과는 affectedRows(바뀐 행 수)·insertId(새 번호)로 확인
핵심 ④	기록이 필요한 주문 취소는 DELETE가 아니라 상태 UPDATE
핵심 ⑤	주문 등록 = INSERT 2개 + 재고 UPDATE — 트랜잭션 한 묶음

다음 주는 경영진의 질문에 SQL로 답하는 종합 실습입니다

다음 주차 예고

14주차 · 경영 데이터 SQL 종합 실습

01

리포트 요청

경영 질문을
SQL로 번역하기

02

매출 리포트

분기별 매출·
베스트셀러·비중

03

고객 분석

미주문 고객·
승급 후보·가입 추이

04

운영 점검

재고 부족·
취소율·직원 실적

13주차 마무리

학습 마무리

조회에 등록·수정·삭제를 더해 다운유통 업무 화면이 완성되었습니다

✓ CRUD와 SQL·HTTP 요청의 대응

✓ UPDATE·DELETE와 affectedRows

✓ 폼 → req.body → INSERT

✓ 트랜잭션으로 묶은 주문 등록

이어서는 활동

실습 문제 w13-1~9 · 총괄보고서 정리